Sprint 2 : JSX, Components, Fragments, and Module Exports

Sprint 2 Goal (අපගේ ඉලක්කය)

- **Concept Explanation (Sinhala):** Sprint 2 එක අවසානයේදී ඔයාට හැමදේම එකම `App.jsx` එකක් අස්සේ ලියන්නේ නැතුව, කුඩා සහ නැවත නැවත පාවිච්චි කරන්න පුළුවන් (reusable) Components විදිහට කඩලා ලස්සන React UI එකක් හදාගන්න පුළුවන් වෙන්න ඕනේ. සරලවම කිව්වොත් React screen එකක් පිරිසිදු, වෙන වෙනම කෑලි වලට කඩන විදිහ තමයි මේ Sprint එකෙන් ඉගෙන ගන්නේ.

1. Current Position After Sprint 1 (Sprint 1 එකෙන් පස්සේ අපේ දැනුම)

- **Concept Explanation (Sinhala):** Sprint 1 එකේදී අපි React වල මූලික පදනම ඉගෙන ගත්තා:
  - React කියන්නේ User Interfaces හදන්න පාවිච්චි කරන JavaScript library එකක්.
  - React apps හදන්නේ නැවත පාවිච්චි කරන්න පුළුවන් Components එකතු කරලා.
  - SPA (Single Page Application) එකකදී වෙන්නේ එකම ප්‍රධාන HTML page එකක් load වෙලා, JavaScript මඟින් screen එකේ තියෙන දේවල් dynamic විදිහට වෙනස් කරන එකයි.
  - Vite පාවිච්චි කරලා අපිට React project එකක් ඉතා ඉක්මනින් හදාගෙන run කරන්න පුළුවන්.
  - අලුත් Vite React project එකක අපි සාමාන්‍යයෙන් මුලින්ම edit කරන්නේ `src/App.jsx` file එකයි.

  Sprint 1 වලදී ඔයා මුළු UI එකම `App.jsx` එක ඇතුලෙම ලියන්න ඇති. නමුත් Sprint 2 එකේදී අපි ඉගෙන ගන්නේ ඒ UI එක තවත් කුඩා files සහ Components වලට වෙන් කරන්නේ කොහොමද කියලයි.

2. What Is JSX?

- **Concept Explanation (Sinhala):** JSX කියන්නේ **JavaScript XML** කියන එකටයි. මේකෙ සම්පූර්ණ නම කටපාඩම් කරන්න අවශ්‍ය නැහැ. ප්‍රධානම අදහස තමයි: JSX මඟින් අපිට JavaScript කේතය ඇතුළේ HTML වගේ පෙනෙන UI කේතයන් ලියන්න ඉඩ ලබා දෙනවා.
- **Code Block:**

```
export default function App() {
  return <h1>Hello React</h1>;
}
```

- **Concept Explanation (Sinhala):** මෙතන තියෙන `<h1>Hello React</h1>` කියන කෑල්ල HTML වගේ පෙනුනට, React file එකක් ඇතුළේදී ඒක JSX වෙනවා. React විසින් screen එකේ පෙන්නන්න ඕනේ දේ විස්තර කරන්න පාවිච්චි කරන්නේ මේ JSX තමයි.

2.1 JSX Is Not Exactly HTML (JSX කියන්නේ කෙලින්ම HTML නෙවෙයි)

- **Concept Explanation (Sinhala):** JSX පෙනුමෙන් HTML වගේ වුණත්, මේක ලියන්නේ JavaScript ඇතුළේ නිසා සුවිශේෂී රීති (rules) ටිකක් තියෙනවා:
  - `class` වෙනුවට `className` පාවිච්චි කරන්න ඕනේ.
  - JavaScript expressions ලියන්න ඕනේ සඟල වරහන් `{}` ඇතුළේ.
  - හැම tag එකක්ම අනිවාර්යයෙන්ම close කරන්න ඕනේ.
  - Components වල නම් පටන් ගන්න ඕනේ Capital අකුරකින් (PascalCase).
  - Component එකකින් return කරන්න පුළුවන් එකම එක parent element එකක් විතරයි.

3. JSX Rule 1: Use className, Not class

- **Concept Explanation (Sinhala):** සාමාන්‍ය HTML වලදී අපි class එකක් දාන්න පාවිච්චි කරන්නේ `class="card"` කියලා වුණත්, React JSX වලදී අපි ලියන්න ඕනේ `className="card"` කියලයි.

  හේතුව තමයි JavaScript වල `class` කියන්නේ reserved word එකක් (වෙනත් කාර්යයක් සඳහා වෙන් කර ඇති වචනයක්). JSX කියන්නේ JavaScript ම නිසා, React වලදී class එකක් හඳුන්වන්න `className` යොදාගන්නවා.
- **Code Block (Correct):**

```
function Card() {
  return <div className="card">This is a card</div>;
}
```

- **Code Block (Incorrect):**

```
function Card() {
  return <div class="card">This is a card</div>;
}
```

4. JSX Rule 2: Put JavaScript Expressions Inside {}

- **Concept Explanation (Sinhala):** JSX ඇතුළේ JavaScript variable එකක අගයක් හෝ value එකක් පෙන්නන්න අවශ්‍ය වුණාම අපි සඟල වරහන් `{}` පාවිච්චි කරනවා.
- **Code Block:**

```
function Greeting() {
  const name = "Sangeeth";
  return <h1>Hello {name}</h1>;
}
```

- **Concept Explanation (Sinhala):** මෙහිදී `name` කියන variable එකේ තියෙන අගය JSX එක ඇතුළට ගිහින් screen එකේ "Hello Sangeeth" කියලා display වෙනවා.

4.1 JSX Can Use Calculations Inside {} (ගණනය කිරීම් සිදු කිරීම)

- **Concept Explanation (Sinhala):** අපිට `{}` ඇතුළේ සරල ගණිත කර්ම පවා සිදු කරලා අගය කෙලින්ම display කරන්න පුළුවන්.
- **Code Block:**

```
function Total() {
  const price = 100;
  const quantity = 3;
  return <p>Total: {price * quantity}</p>;
}
```

- **Concept Explanation (Sinhala):** මෙතනදී React විසින් `price * quantity` (100 * 3) ගණනය කරලා "Total: 300" කියලා screen එකේ පෙන්වනවා.

4.2 JSX Can Use Conditional Expressions Inside {} (කොන්දේසි පරීක්ෂා කිරීම)

- **Concept Explanation (Sinhala):** අපිට ternary operator (`? :`) එක පාවිච්චි කරලා `{}` ඇතුළේ conditional expressions ලියන්න පුළුවන්.
- **Code Block:**

```
function AgeMessage() {
  const age = 20;
  return <p>{age >= 18 ? "Adult" : "Minor"}</p>;
}
```

- **Concept Explanation (Sinhala):** මෙහිදී `age` එක 18 ට වඩා වැඩි හෝ සමාන නම් "Adult" කියලත්, නැත්නම් "Minor" කියලත් screen එකේ පෙන්වනවා.

4.3 Expressions vs Statements

- **Concept Explanation (Sinhala):** JSX `{}` ඇතුළේ ලියන්න පුළුවන් JavaScript expressions විතරයි. Expression එකක් කියන්නේ අවසානයේදී අගයක් (value එකක්) ලබාදෙන කේතයකටයි.

  නමුත් `if` statement එකක් කියන්නේ statement එකක් මිස expression එකක් නෙවෙයි. ඒ නිසා කෙලින්ම `{}` ඇතුළේ `if` statement එකක් ලියන්න බැහැ.
- **Code Block (Good):**

```
<p>{age >= 18 ? "Adult" : "Minor"}</p>
```

- **Code Block (Bad):**

```
<p>{if (age >= 18) "Adult"}</p>
```

- **Concept Explanation (Sinhala):** මතක තියාගන්න, `{}` ඇතුළේ ලියන්න පුළුවන් variables, calculations, function calls, හෝ ternary expressions විතරයි.

5. JSX Rule 3: Every Tag Must Close (හැම tag එකක්ම වැසිය යුතුය)

- **Concept Explanation (Sinhala):** සාමාන්‍ය HTML වලදී සමහර tags වැහුවේ නැතත් වැඩ කරාට, JSX වලදී හැම tag එකක්ම අනිවාර්යයෙන්ම close කරන්න ඕනේ. Single tags (self-closing tags) සඳහා අවසානයට `/` එකක් යෙදිය යුතුමයි.
- **Code Block:**

```
// HTML වල මෙහෙම ලිව්වට:
// <img src="logo.png">

// JSX වලදී අනිවාර්යයෙන්ම මෙහෙම ලියන්න ඕනේ:
<img src="logo.png" alt="Logo" />

// Inputs සඳහාද එසේමයි:
<input type="text" />

// Custom React Components සඳහාද එසේමයි:
<Header />
```

6. What Is a React Component?

- **Concept Explanation (Sinhala):** React Component එකක් කියන්නේ සරලවම UI එකක් return කරන JavaScript function එකකටයි.

  එය Component එකක් වෙන්න කරුණු 4ක් සම්පූර්ණ වෙන්න ඕනේ:
  1. ඒක JavaScript function එකක් වෙන්න ඕනේ.
  2. ඒකෙ නම පටන් ගන්න ඕනේ Capital අකුරකින් (PascalCase).
  3. ඒකෙන් JSX return කරන්න ඕනේ.
  4. ඒක අපිට custom HTML tag එකක් වගේ පාවිච්චි කරන්න පුළුවන් වෙන්න ඕනේ.
- **Code Block:**

```
function Header() {
  return <h1>My Website</h1>;
}

function App() {
  return (
    <main>
      <Header />
      <p>Welcome to my app.</p>
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** මෙතන `Header` කියන්නේ අපිට නැවත නැවත පාවිච්චි කරන්න පුළුවන් UI කෑල්ලක් (reusable UI piece).

7. Why Components Are Useful (Components වැදගත් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** ලොකු UI එකක් කුඩා කොටස් වලට කඩාගන්න Components අපිට උදව් වෙනවා. මුළු page එකම එකම file එකක හදනවා වෙනුවට, එක් එක් කාර්යයට වෙන වෙනම Components හදන්න පුළුවන්.

  උදාහරණයක් ලෙස:
  - `Header` : Page එකේ ඉහළ කොටස පෙන්වයි.
  - `AboutBox` : කෙනෙක් ගැන තොරතුරු පෙන්වයි.
  - `SkillBadge` : Skill එකක් පෙන්වයි.
  - `Footer` : Page එකේ පහළ කොටස පෙන්වයි.

  මේ නිසා කේතය:
  - කියවන්න පහසුයි (Easier to read).
  - නැවත පාවිච්චි කරන්න පුළුවන් (Easier to reuse).
  - ලෙහෙසියෙන් debug කරන්න සහ fix කරන්න පුළුවන්.
  - පිළිවෙලට තියාගන්න පුළුවන් (Easier to organize).

8. Component Names Must Start With a Capital Letter

- **Concept Explanation (Sinhala):** React වලදී custom components හඳුනාගන්නේ ඒවායේ නම පටන් ගන්නා Capital අකුරෙනි. නම lowercase (කුඩා අකුරෙන්) තිබුණොත් React හිතන්නේ ඒක සාමාන්‍ය HTML tag එකක් (උදා: `<div>` , `<p>`) කියලයි.
- **Code Block (Correct):**

```
function ProfileCard() {
  return <div>Profile</div>;
}
// භාවිතය: <ProfileCard />
```

- **Code Block (Incorrect):**

```
function profileCard() {
  return <div>Profile</div>;
}
// භාවිතය: <profileCard />  <-- මෙක වැරදියි!
```

- **Concept Explanation (Sinhala):** Components සඳහා නිතරම **PascalCase** පාවිච්චි කරන්න (උදා: `Header` , `ProfileCard` , `SkillBadge`).

9. Using Components Inside Other Components (Component Composition)

- **Concept Explanation (Sinhala):** අපිට එක Component එකක් ඇතුළේ තවත් Component එකක් පාවිච්චි කරන්න පුළුවන්. කුඩා Components කිහිපයක් එකතු කරලා ලොකු UI එකක් නිර්මාණය කිරීමට **Component Composition** කියලා කියනවා.
- **Code Block:**

```
function Header() {
  return <h1>My Profile</h1>;
}

function App() {
  return (
    <main>
      <Header />
      <p>Welcome to my profile page.</p>
    </main>
  );
}
```

10. Example: Before Components (Components වලට කඩන්න කලින්)

- **Concept Explanation (Sinhala):** හැමදේම එකම `App.jsx` එක ඇතුළේ ලිව්වම කේතය සංකීර්ණ වෙනවා.
- **Code Block:**

```
export default function App() {
  return (
    <main>
      <h1>My Profile</h1>
      <section>
        <h2>About Me</h2>
        <p>I am learning React.</p>
      </section>
      <section>
        <h2>Skills</h2>
        <p>JavaScript, React, CSS</p>
      </section>
    </main>
  );
}
```

11. Example: After Splitting Into Components (Components වලට කැඩූ පසු)

- **Concept Explanation (Sinhala):** ඉහත කේතයම අපිට වෙන වෙනම Components වලට කඩලා ලස්සනට ලියන්න පුළුවන්.
- **Code Block:**

```
function Header() {
  return <h1>My Profile</h1>;
}

function AboutBox() {
  return (
    <section>
      <h2>About Me</h2>
      <p>I am learning React.</p>
    </section>
  );
}

function SkillsBox() {
  return (
    <section>
      <h2>Skills</h2>
      <p>JavaScript, React, CSS</p>
    </section>
  );
}

export default function App() {
  return (
    <main>
      <Header />
      <AboutBox />
      <SkillsBox />
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** දැන් කේතය ඉතා පිරිසිදුයි. `Header` එක හෙඩින් එක බලාගන්නවා, `AboutBox` එක විස්තරය බලාගන්නවා, සහ `App` එක මුළු page එකේම ව්‍යුහය පාලනය කරනවා.

12. The One Parent Element Rule (එක් parent element එකක් පමණක් තිබීමේ රීතිය)

- **Concept Explanation (Sinhala):** Component එකකින් return කරන්න පුළුවන් එකම එක parent element එකක් විතරයි. Sibling elements (සමකාලීන මට්ටමේ පවතින elements) කිහිපයක් එකවර return කරන්න බැහැ.
- **Code Block (Incorrect):**

```
function App() {
  return (
    <h1>Hello</h1>
    <p>Welcome</p> // <-- වැරදියි! Parent කෙනෙක් නැහැ.
  );
}
```

12.1 Fix With a div (div එකක් මඟින් විසඳීම)

- **Concept Explanation (Sinhala):** අපිට මේවා එක `<div>` එකක් ඇතුළට දාලා return කරන්න පුළුවන්.
- **Code Block:**

```
function App() {
  return (
    <div>
      <h1>Hello</h1>
      <p>Welcome</p>
    </div>
  );
}
```

12.2 Better Fix With Semantic HTML (අර්ථවත් HTML භාවිතය)

- **Concept Explanation (Sinhala):** හැම තැනටම `div` දානවාට වඩා Semantic HTML elements (`main` , `section` , `header` , `footer` ආදිය) පාවිච්චි කිරීම වඩාත් ගැලපෙනවා.
- **Code Block:**

```
function App() {
  return (
    <main>
      <h1>Hello</h1>
      <p>Welcome</p>
    </main>
  );
}
```

13. What Are Fragments?

- **Concept Explanation (Sinhala):** සමහර වෙලාවට අපිට HTML එකට අනවශ්‍ය විදිහට අලුත් `div` එකක් එකතු කරන්නේ නැතුව elements කිහිපයක් return කරන්න අවශ්‍ය වෙනවා. ඒකට තියෙන විසඳුම තමයි **React Fragments** කියන්නේ.

  Fragment එකක් කියන්නේ HTML DOM එකට අමතර element එකක් එකතු නොකර, JSX elements සමූහයක් එකට එකතු කරන්න පුළුවන් නොපෙනෙන wrapper (invisible wrapper) එකක්.
- **Code Block:**

```
function Greeting() {
  return (
    <>
      <h1>Hello</h1>
      <p>Welcome to React</p>
    </>
  );
}
```

- **Concept Explanation (Sinhala):** මෙහිදී `<>` සහ `</>` කියන සලකුණෙන් Fragment එකක් නිරූපණය වෙනවා.

13.1 Long Fragment Syntax

- **Concept Explanation (Sinhala):** අපිට Fragment එක මේ විදිහට ලියන්නත් පුළුවන්. නමුත් බහුතරයක් දෙනා කෙටි ක්‍රමය (`<></>`) පාවිච්චි කරනවා.
- **Code Block:**

```
import { Fragment } from "react";

function Greeting() {
  return (
    <Fragment>
      <h1>Hello</h1>
      <p>Welcome</p>
    </Fragment>
  );
}
```

14. Files and Components

- **Concept Explanation (Sinhala):** කුඩා උදාහරණ වලදී අපි ඔක්කොම එකම file එකේ ලිව්වට, සැබෑ React project එකකදී අපි කරන්නේ හැම Component එකක්ම වෙන වෙනම files වලට කඩන එකයි.

  සාමාන්‍යයෙන් අපි මේවා දාන්නේ `components` කියන folder එක ඇතුළටයි.

```
src/
├── App.jsx
└── components/
    ├── Header.jsx
    ├── ProfileCard.jsx
    └── SkillBadge.jsx
```

15. What Is an Export?

- **Concept Explanation (Sinhala):** `export` එකක් මඟින් එක file එකක තියෙන කේතයක් (Component එකක් වගේ දෙයක්) තවත් file එකකට පාවිච්චි කරන්න අවස්ථාව ලබා දෙනවා (Make it available).
- **Code Block:**

```
export default function Header() {
  return <h1>My React App</h1>;
}
```

16. What Is an Import?

- **Concept Explanation (Sinhala):** `import` එකකින් කරන්නේ වෙනත් file එකකින් export කරපු කේතයක් අපේ වත්මන් file එක ඇතුළට ගෙන්වා ගැනීමයි.
- **Code Block:**

```
import Header from "./components/Header";
```

17. Default Exports

- **Concept Explanation (Sinhala):** එක file එකකින් කරන්න පුළුවන් එකම එක **default export** එකක් විතරයි. සාමාන්‍යයෙන් file එකක තියෙන ප්‍රධානම Component එක අපි default export කරනවා.

  Default export එකක් import කිරීමේදී සඟල වරහන් `{}` පාවිච්චි කරන්නේ නැහැ.
- **Code Block (Header.jsx):**

```
// src/components/Header.jsx
export default function Header() {
  return <h1>My React App</h1>;
}
```

- **Code Block (App.jsx):**

```
// src/App.jsx
import Header from "./components/Header"; // සඟල වරහන් නැත!

export default function App() {
  return (
    <main>
      <Header />
    </main>
  );
}
```

18. Named Exports

- **Concept Explanation (Sinhala):** එක file එකකින් අපිට අවශ්‍ය තරම් **named exports** ප්‍රමාණයක් කරන්න පුළුවන්.

  Named export එකක් import කිරීමේදී අනිවාර්යයෙන්ම සඟල වරහන් `{}` පාවිච්චි කළ යුතුමයි.
- **Code Block (ProfileCard.jsx):**

```
// src/components/ProfileCard.jsx
export function ProfileCard() {
  return (
    <section>
      <h2>Sangeeth</h2>
      <p>React learner</p>
    </section>
  );
}
```

- **Code Block (App.jsx):**

```
import { ProfileCard } from "./components/ProfileCard"; // සඟල වරහන්
අනිවාර්යයි!
```

18.1 Multiple Named Exports in One File (එකම file එකකින් කිහිපයක් export කිරීම)

- **Code Block (Animals.jsx):**

```
// src/components/Animals.jsx
export function Cat() {
  return <p>Cat</p>;
}
export function Dog() {
  return <p>Dog</p>;
}
export function Bird() {
  return <p>Bird</p>;
}
```

- **Code Block (App.jsx):**

```
import { Cat, Dog, Bird } from "./components/Animals"; // සඟල වරහන් ඇතුළේ
ලියයි

function App() {
  return (
    <main>
      <Cat />
      <Dog />
      <Bird />
    </main>
  );
}
```

19. Default Export vs Named Export (වෙනස්කම්)

- **Concept Explanation (Sinhala):**
  - **Default Export:**
    - එක් file එකකට තිබිය හැක්කේ එකක් පමණි.
    - Import කිරීමේදී සඟල වරහන් `{}` අවශ්‍ය නොවේ.
    - සාමාන්‍යයෙන් යොදාගන්නේ file එකෙහි ඇති ප්‍රධානම component එක සඳහාය.
  - **Named Export:**
    - එක් file එකකට ඕනෑතරම් තිබිය හැක.
    - Import කිරීමේදී සඟල වරහන් `{}` අනිවාර්යයෙන්ම අවශ්‍ය වේ.
    - සාමාන්‍යයෙන් යොදාගන්නේ file එක තුළ විවිධ කුඩා components කිහිපයක් හෝ utility functions කිහිපයක් ඇති විටය.

20. Common Import Mistakes (පටලවා ගන්නා තැන්)

Mistake 1: Default export එකක් සඟල වරහන් සහිතව import කිරීම

- **Code Block (Incorrect):**

```
import { Header } from "./Header"; // වැරදියි! Default export එකක් නිසා {}
අවශ්‍ය නැත.
```

- **Code Block (Correct):**

```
import Header from "./Header";
```

Mistake 2: Named export එකක් සඟල වරහන් නැතුව import කිරීම

- **Code Block (Incorrect):**

```
import ProfileCard from "./ProfileCard"; // වැරදියි! Named export එකක් නිසා
{} අවශ්‍ය වේ.
```

- **Code Block (Correct):**

```
import { ProfileCard } from "./ProfileCard";
```

21. Complete Sprint 2 Example Project (සම්පූර්ණ උදාහරණය)

21.1 Header.jsx (Default Export)

- **Code Block:**

```
export default function Header() {
  const title = "My React Profile";
  return (
    <header>
      <h1>{title}</h1>
      <p>I am learning components, JSX, and imports.</p>
    </header>
  );
}
```

21.2 ProfileCard.jsx (Named Export)

- **Code Block:**

```
export function ProfileCard() {
  return (
    <section className="card">
      <h2>Sangeeth</h2>
      <p>Frontend learner building React step by step.</p>
    </section>
  );
}
```

21.3 SkillBadge.jsx (Named Export)

- **Code Block:**

```
export function SkillBadge() {
  return <span className="skill-badge">React Beginner</span>;
}
```

21.4 App.jsx (සම්බන්ධ කරන ප්‍රධාන file එක)

- **Code Block:**

```
import Header from "./components/Header";
import { ProfileCard } from "./components/ProfileCard";
import { SkillBadge } from "./components/SkillBadge";

export default function App() {
  return (
    <main>
      <Header />
      <section>
        <h2>About</h2>
        <ProfileCard />
      </section>
      <section>
        <h2>Current Skill</h2>
        <SkillBadge />
      </section>
    </main>
  );
}
```

22. Production-Friendly Beginner Structure (වෘත්තීය මට්ටමේ Folder ව්‍යුහය)

- **Concept Explanation (Sinhala):** Project එක ලොකු වෙනකොට හැම file එකක්ම එකම තැන දාන්නේ නැතුව මෙන්න මේ විදිහට folders වලට වෙන් කරනවා:
  - `components/` : නැවත නැවත පාවිච්චි කරන්න පුළුවන් UI කෑලි (උදා: `Button.jsx` , `Card.jsx` , `Navbar.jsx`).
  - `pages/` : සම්පූර්ණ screen එකක් හෝ route එකක් නියෝජනය කරන components (උදා: `Home.jsx` , `Products.jsx` , `Login.jsx`).
  - `hooks/` : Custom React hooks සඳහා.
  - `services/` : API හෝ backend එකත් එක්ක කරන ගනුදෙනු බලාගන්න කේත (උදා: `api.js`).
  - `context/` : Shared app state providers සඳහා.
  - `utils/` : පොදු helper functions සඳහා (උදා: `formatPrice.js`).
  - `assets/` : පින්තූර, icons හෝ static files සඳහා.

23. Reusable Components vs Page-Level Components

- **Concept Explanation (Sinhala):**
  - **Reusable Component:** කුඩා UI කොටසක් වන අතර එය විවිධ ස්ථාන වල කිහිප වතාවක් පාවිච්චි කළ හැක (උදා: `Button` , `Card` , `SkillBadge`). මේවා `components/` folder එකට දමනු ලැබේ.
  - **Page Component:** සම්පූර්ණ screen එකක් නියෝජනය කරයි (උදා: `HomePage` , `LoginPage` , `DashboardPage`). මේවා `pages/` folder එකට දමනු ලැබේ.

24. Debug Practice (වැරදි කේතයන් නිවැරදි කිරීම)

සදොස් කේතය (Broken Code):

- **Code Block:**

```
function App() {
  return <h1>Hello</h1>
  <p>React</p>
}

function profilecard() {
  return <div class="card">Card</div>
}

import { Header } from "./Header";
```

ඇති වැරදි සහ ඒවා නිවැරදි කරන ආකාරය:

- **Concept Explanation (Sinhala):**
  1. **වැරැද්ද 1 (Parent එකක් නොමැතිකම):** `App` එකෙන් elements දෙකක් කෙලින්ම return කරන්න උත්සාහ කර ඇත. ඒ සඳහා Fragment (`<></>`) එකක් භාවිතා කළ යුතුය.
  2. **වැරැද්ද 2 (කුඩා අකුරු භාවිතය):** `profilecard` යන්නෙහි පළමු අකුර කුඩා අකුරක් වේ. React component එකක් නිසා එය `ProfileCard` ලෙස Capital විය යුතුය.
  3. **වැරැද්ද 3 (`class` භාවිතය):** JSX වල `class` වෙනුවට `className` භාවිතා කළ යුතුය.
  4. **වැරැද්ද 4 (වැරදි Import එක):** `Header` එක export කර ඇත්තේ default export එකක් ලෙස නම්, import කිරීමේදී සඟල වරහන් `{}` භාවිත නොකළ යුතුය.

නිවැරදි කරන ලද කේතය (Corrected Code):

- **Code Block:**

```
import Header from "./Header";

function ProfileCard() {
  return <div className="card">Card</div>;
}

export default function App() {
  return (
    <>
      <Header />
      <h1>Hello</h1>
      <p>React</p>
      <ProfileCard />
    </>
  );
}
```

25. Sprint 2 Challenge Task (Animals.jsx)

- **Concept Explanation (Sinhala):** එකම file එකක් ඇතුළේ default export එකක් සහ named exports කිහිපයක් එකට මිශ්‍ර කර පාවිච්චි කරන ආකාරය පුහුණු වීම සඳහා මෙම අභ්‍යාසය උපකාරී වේ.
- **Code Block (Animals.jsx):**

```
// src/components/Animals.jsx
export default function AnimalTitle() {
  return <h2>Animals</h2>;
}

export function Cat() {
  return <p>Cat</p>;
}

export function Dog() {
  return <p>Dog</p>;
}
```

- **Code Block (App.jsx):**

```
import AnimalTitle, { Cat, Dog } from "./components/Animals"; // Default
සහ Named එකට Import කිරීම

export default function App() {
  return (
    <main>
      <AnimalTitle />
      <Cat />
      <Dog />
    </main>
  );
}
```

26. Revision Checkpoint Questions (ප්‍රශ්න සහ පිළිතුරු)

- **Concept Explanation (Sinhala):**
  - **ප්‍රශ්නය 1: JSX කියන්නේ මොකක්ද?**
    - **පිළිතුර:** JSX කියන්නේ JavaScript ඇතුළේ HTML වගේ පෙනෙන UI කේතයන් ලියන්න ඉඩ සලසන විශේෂ syntax එකක්.
  - **ප්‍රශ්නය 2: JSX වලදී class වෙනුවට className පාවිච්චි කරන්නේ ඇයි?**
    - **පිළිතුර:** මොකද `class` කියන්නේ JavaScript වල වෙනත් වැඩකට වෙන් කරලා තියෙන (reserved) වචනයක් නිසා.
  - **ප්‍රශ්නය 3: React components වල නම අනිවාර්යයෙන්ම Capital අකුරකින් පටන් ගන්න ඕනේ ඇයි?**
    - **පිළිතුර:** React විසින් කුඩා අකුරෙන් පටන් ගන්නා tags සාමාන්‍ය HTML elements ලෙසත්, Capital අකුරෙන් පටන් ගන්නා tags custom components ලෙසත් හඳුනා ගන්නා බැවිනි.
  - **ප්‍රශ්නය 4: Fragments වලින් විසඳන ප්‍රශ්නය මොකක්ද?**
    - **පිළිතුර:** HTML DOM එකට අනවශ්‍ය ලෙස අමතර elements එකතු කරන්නේ නැතුව, elements කිහිපයක් එක parent කෙනෙක් යටතේ return කිරීමට Fragments උපකාරී වේ.
  - **ප්‍රශ්නය 5: Named export එකක් import කරන්නේ කොහොමද?**
    - **පිළිතුර:** සඟල වරහන් `{}` ඇතුළත ලියා import කළ යුතුය (උදා: `import { ProfileCard } from "./ProfileCard"`).
  - **ප්‍රශ්නය 6: Default export එකක් import කරන්නේ කොහොමද?**
    - **පිළිතුර:** සඟල වරහන් `{}` රහිතව import කළ යුතුය (උදා: `import Header from "./Header"`).
  - **ප්‍රශ්නය 7: Reusable components දාන්න ඕනේ මොන folder එකටද?**
    - **පිළිතුර:** `components/` folder එකට.
  - **ප්‍රශ්නය 8: Route-level pages දාන්න ඕනේ මොන folder එකටද?**
    - **පිළිතුර:** `pages/` folder එකට.
  - **ප්‍රශ්නය 9: `services/` folder එකට දාන්නේ මොනවාද?**
    - **පිළිතුර:** API Calls සහ backend සන්නිවේදන කටයුතු සිදු කරන කේතයන්ය.
  - **ප්‍රශ්නය 10: `utils/` folder එකට දාන්නේ මොනවාද?**
    - **පිළිතුර:** පොදුවේ ඕනෑම තැනක පාවිච්චි කළ හැකි සරල helper functions ය (උදා: මිල ගණන් format කරන functions).

27. Mini Revision Sheet (කෙටි සටහන)

- **Concept Explanation (Sinhala):**
  - **JSX:** JavaScript ඇතුළේ ලියන UI-like syntax එකක්.
  - **Component:** UI එකක් return කරන JavaScript function එකක්.
  - **className:** HTML වල `class` එක සඳහා JSX වල යොදන ආදේශකය.
  - **`{}` in JSX:** JavaScript expressions ලිවීම සඳහා යොදා ගනී.
  - **Fragment:** elements කිහිපයක් වට කිරීමට යොදන නොපෙනෙන wrapper එකක් (`<></>`).
  - **Default Export:** file එකකින් කරන ප්‍රධානම export එකයි (Import with no braces).
  - **Named Export:** file එකකින් කරන අමතර export එකක් හෝ කිහිපයකි (Import with braces `{}`).
  - **components/:** නැවත නැවත පාවිච්චි කළ හැකි කුඩා UI කොටස්.
  - **pages/:** සම්පූර්ණ screens හෝ routes.